SIWY - regulamin projektów

01.10.2026

§ 1 Informacje ogólne

1) **Projekt może dotyczyć**:
   - eksperymentalnych badań naukowych lub odtworzenia eksperymentu z istniejącego artykułu z dziedziny AI / data science
   - wytworzenia nowego narzędzia, frameworku, biblioteki
   - budowy aplikacji webowej
   - rozbudowy istniejącego narzędzia, w szczególności z poprzedniej edycji SIWY
2) **Projekt musi cechować się**:
   - wysoką jakością kodu
   - reprodukowalnością
   - rzetelnością
   - zastosowaniem nietrywialnych narzędzi
3) Do projektu należy dostarczyć dokumentację, instrukcję użytkowania oraz adekwatne do tematu projektu testy.
4) **Pierwszym etapem projektu jest dostarczenie pisemnego design proposal** („projekt projektu"). Design proposal zawiera:
   - harmonogram projektu z podziałem na tygodnie i planowanymi postępami projektu per tydzień
   - bibliografię
   - planowany zakres eksperymentów
   - planowaną funkcjonalność programu
   - planowany stack technologiczny

   Design proposal może zostać dostarczony w formacie `.md` albo `.pdf`, najlepiej zamieścić go w repozytorium projektu. Harmonogram należy czytelnie rozplanować i opisać wg dat, a nie tylko numerów tygodni.

   W harmonogramie projektu należy również uwzględnić potencjalny czas potrzebny na trenowanie modeli i przewidywane zapotrzebowanie na zasoby obliczeniowe.
5) W kolejnym etapie należy zademonstrować adekwatny do tematu projektu **funkcjonalny prototyp projektu, postęp analizy literaturowej, konfigurację środowiska eksperymentalnego.**
6) Analiza literatury **musi** uzasadniać dalsze działania i być udokumentowana, np. w postaci tabeli w `.md` zawierającej linki do artykułów, autorski komentarz, informację o tym czy dostępny jest kod i pre-trenowane modele, jakie metryki są zawarte w ewaluacji, jakie zasoby obliczeniowe były wykorzystane przez autorów itp.

   Brak dokumentacji analizy literaturowej jest traktowany jako brak analizy literaturowej.
7) Pracę nad projektem, ewentualne odchylenia od harmonogramu, pośrednie wyniki eksperymentów itp. należy na bieżąco dokumentować w prosty sposób (np. krótkie notatki w `.md` w repozytorium, issue tracker itp.) tak, aby być w stanie zreferować bieżące postępy na zasadzie „weekly standup". W ciągu semestru, między demonstracją prototypu a oddaniem projektu, nastąpi co najmniej jedno zapytanie o zreferowanie postępów na takiej zasadzie.
8) Do projektu należy dostarczyć:
   - 3-5min filmik demonstrujący projekt „w pigułce".
   - Artykuł naukowy opisujący projekt i uzyskane wyniki.
9) Na ostatnim wykładzie odbywają się publiczne prezentacje projektów.
10) Brak design proposal, prototypu, instrukcji użytkowania, dokumentacji, adekwatnych do projektu testów, filmiku podsumowującego lub finalnej prezentacji skutkuje niezaliczeniem projektu.
11) Ocena z projektu może, lecz nie musi być jednakowa dla wszystkich członków zespołu. Brak wkładu w projekt skutkuje niezaliczeniem projektu. Za przejawy braku wkładu w projekt uznawane są w szczególności brak kontrybucji kodu do projektu (w postaci commitów) i brak pisemnych opracowań pracy badawczej.

§ 2 Szczegóły technologiczne, wymagania i zalecenia

Ogólne wymagania i zalecenia technologiczne

- autoformatter (np. black, ruff), linter (np. ruff)
- środowisko wirtualne (np. venv, uv, conda, poetry)
- oskryptowane budowanie, testowanie, uruchamianie aplikacji (np. just, tradycyjne skrypty powłoki, skrypty w Pythonie - argparse, typer, click)
- dokumentacja
- instrukcja użytkowania
- styl zgodny z PEP8 (ze zwiększonym limitem długości linii)
- rzetelne korzystanie z repozytorium, np. conventional commits https://www.conventionalcommits.org/en/v1.0.0/

Wymagania: projekty AI / data science / eksperymentalne

- niemutowalność oryginalnych danych
- rzetelna struktura projektu (np. cookiecutter, copier)
- śledzenie eksperymentów (np. Tensorboard / W&B)
- konfiguracja odseparowana od kodu wykonawczego
- eksport kodu z notatników do kodu bibliotecznego
- korzystanie z frameworków odpowiednich dla danego zagadnienia (np. Lightning, Huggingface)

Wymagania: aplikacje webowe

- warstwowa struktura aplikacji
- https://12factor.net/ https://12factor.net/pl/
- wykorzystanie bazy danych
- cache, kolejki (np. Redis, RabbitMQ - frameworki Dramatiq, Celery)
- mechanizmy zbierania logów i monitorowania aplikacji (np. logging, loguru, Sentry)
- konteneryzacja
- testy automatyczne: jednostkowe, wydajnościowe
- specyfikacja OpenAPI (Swagger)

Wymagania: nowe frameworki, biblioteki

- semantic versioning: https://semver.org/
- budowa paczki (np. uv, pdm)
- mkdocs, sphinx
- testy automatyczne na co najmniej kilku wersjach interpretera (np. tox)